Bare Metal/RTOS and MSP432 Summary

ECE 2564, Embedded Systems

Fall 2025

Overview

- Bare metal programming versus Real Time Operating System (RTOS)
- Some additional MSP432 hardware features

Motivation

- Should the code run directly on the MSP432?
- Or should the code run with an operating system (OS)?

- How do we ensure that real-time constraints are satisfied?
  - Need to meet real-time constraints
- If the real-time constraints are not satisfied, will that render the application code useless?

Bare-Metal Programming

- What happens when you flash your project onto the MSP432?
  - The executable code travels through the USB cable and the debugger chip
  - It arrives at the flash memory of the MSP432
  - The Program Counter is reset to the beginning of your code
  - The processor starts running the instructions in your program
- In the steps described above, your code is the *only* code running on the MSP432
  - No other program interferes with the running of your code
  - This is called *bare-metal* programming

Operating System

- An alternative to bare-metal is to use an operating system
  - In its simplest form, the OS is a *scheduler* that decides what part of code to run next
  - The OS could frequently preempt your code and perform other tasks
- An operating system that serves real-time applications is called a Real-Time Operating System (RTOS)
- Some RTOS options:
  - A simple scheduler that you write, possibly a few hundred lines of code
  - Various open-source, free RTOS options are available
    - Provided by other programmers (maybe you)
    - Provided by companies like TI to support their chip-sets
  - Commercial Off-the-Shelf (CTOS) licensed RTOS
  - **https://www.lynx.com/embedded-systems-learning-center/most-popular-real-time-operating-systems-rtos**

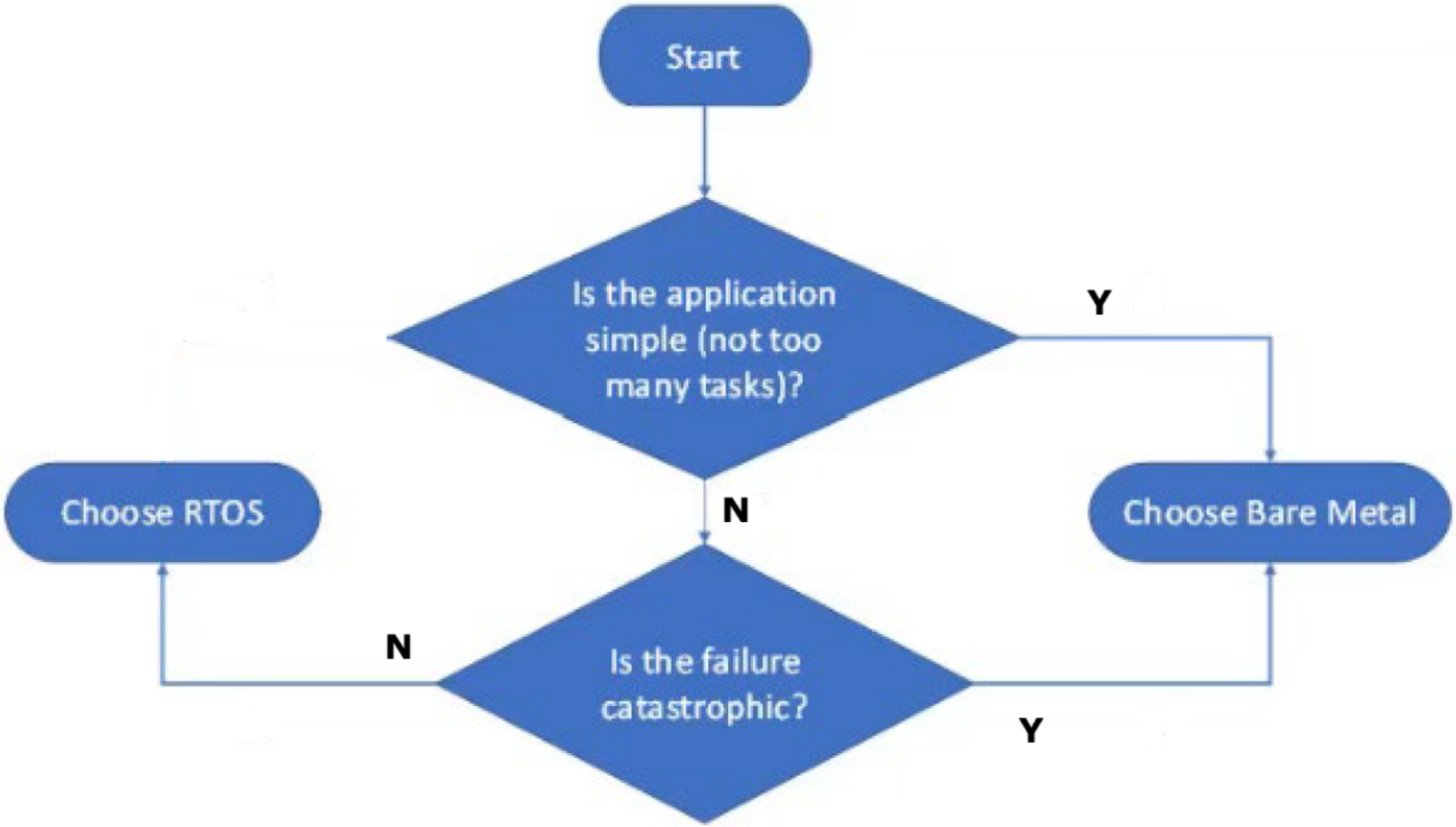
Bare-Metal/OS Examples

- Suppose you want to interface with each of the following devices:
  - LEDs, buttons, timers, joystick
- Option 1: Bare-metal – the programmer writes the entire program
  - You decide how and when the various peripherals are checked, updated, etc.
- Option 2: Let the RTOS take care of scheduling the individual parts of code
  - You could write a separate "thread" of code for each peripheral
  - At initialization, launch all of the threads
  - The RTOS can automatically let thread #1 for a while, and then let thread #2 run for a while, and then ...

Bare-Metal Versus RTOS

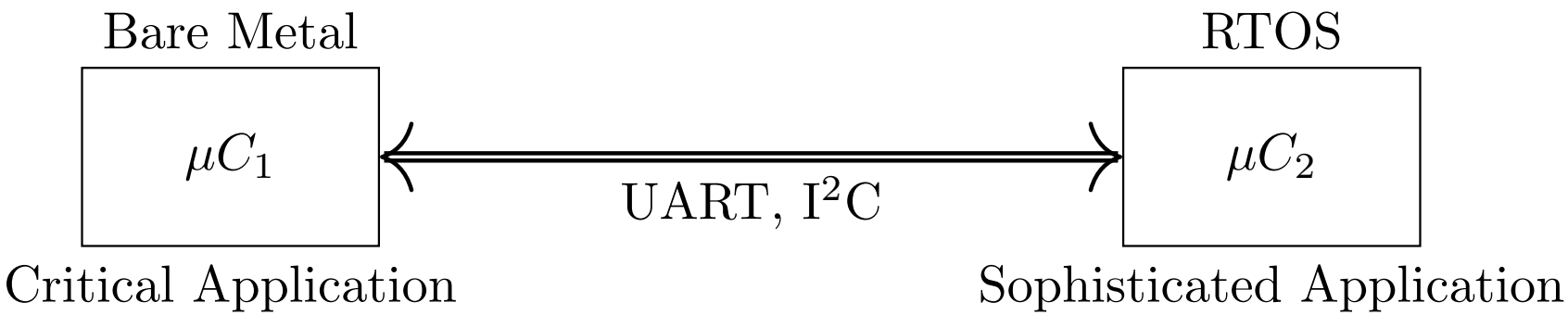
	Bare Metal	RTOS
Full control over program execution	Wins	
Easier to test/debug	Wins (Debugger works!)	(single threads are relatively easy to test and debug) (Debugging across multiple threads is very difficult)
Faster time-to-market	Wins for simple applications	Wins for complicated applications
More deterministic code behavior (useful for critical applications)	Wins	
Easier to support desktop-style I/O, sophisticated user interfaces		Wins

Choosing Between Bare-Metal and RTOS: Decision Tree



Hybrid Embedded System

- Combines multiple embedded systems: Bare-metal and RTOS
  - Example: Embedded system on a rocket
  - May include more than one CPU



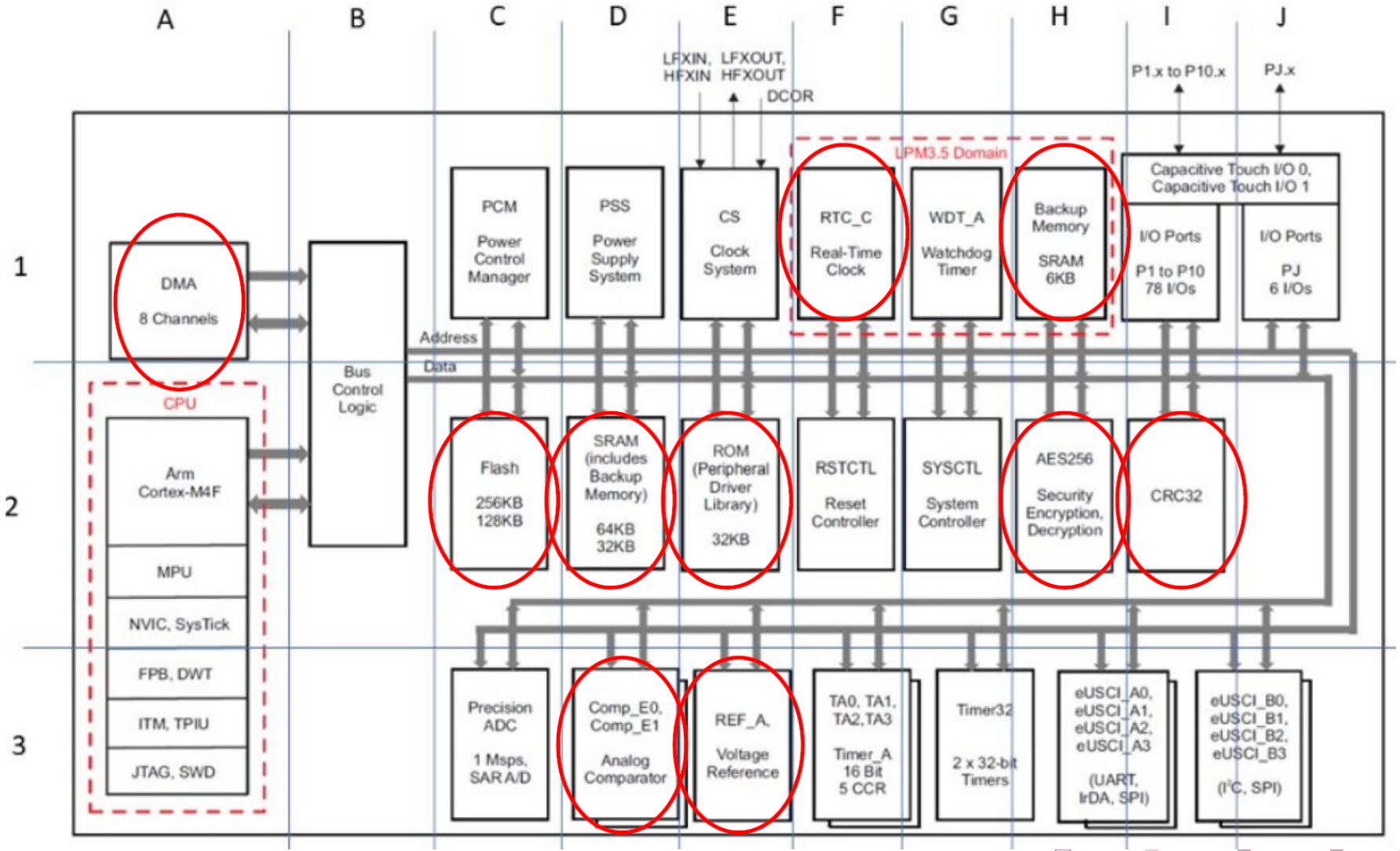
Summary: Bare-Metal Programming versus RTOS

- Pros and cons of both approaches
- How to determine which one to use
- Can learn more about OS/schedulers
  - OS courses (general or Linux-based)
  - Some companies that develop RTOS products provide training

MSP432 Hardware Wrapup: Outline

- A brief look at some remaining hardware subsystems in the MSP432
- What has been covered?
  - SPI, I^2C, UART
  - GPIO
  - ADC
  - Timer32, Timer_A, watchdog timer
  - Memory
  - Interrupt controller
- Modules that we will NOT discuss:
  - Power supply, power control, clock system
  - Reset control, system control, bus control

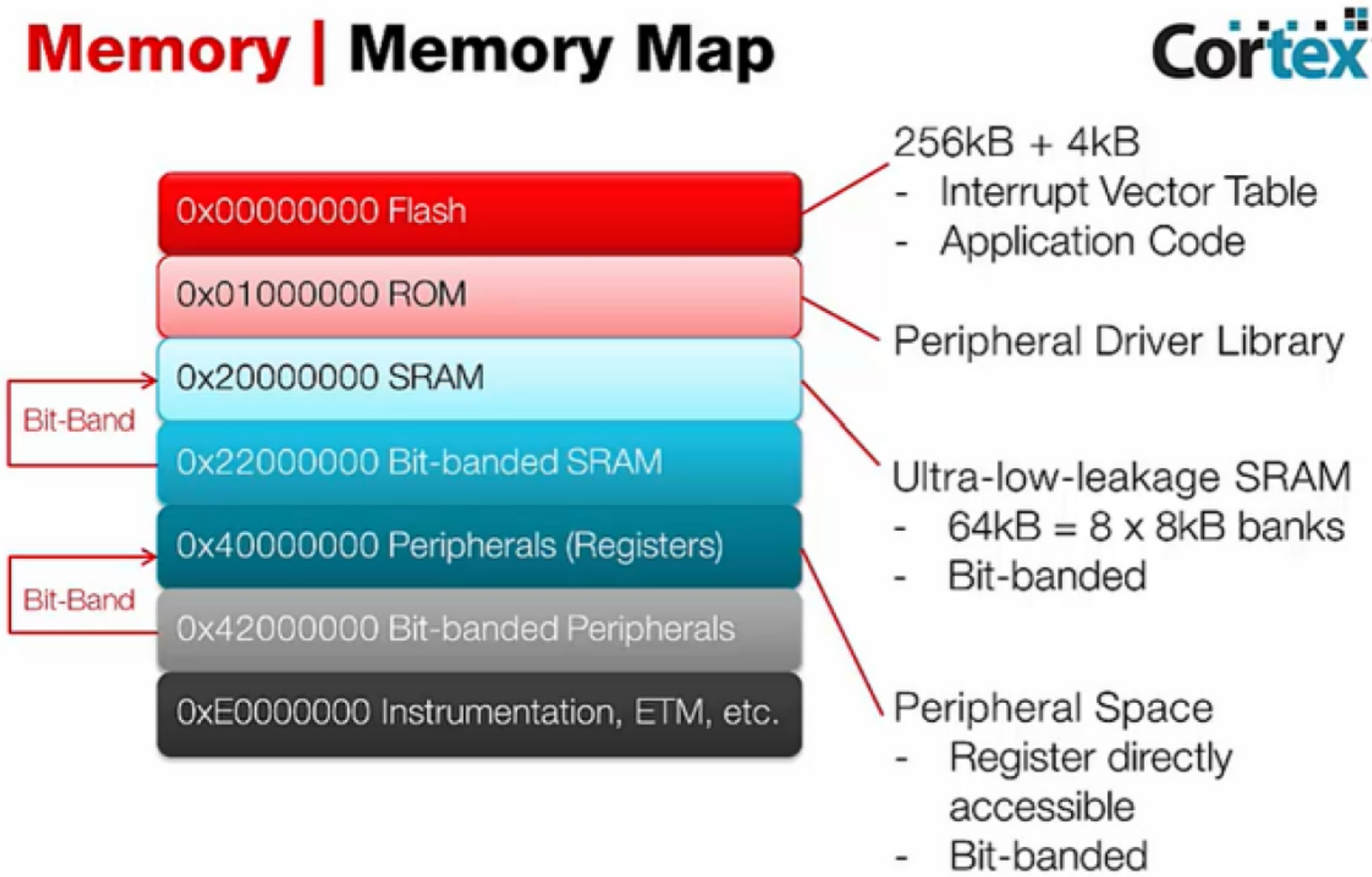
MSP Functional Block Diagram: Additional Features



Memory Blocks

- RAM: Random Access Memory
  - SRAM: Static RAM – fast, expensive, easy-to-write, volatile
  - DRAM: Dynamic RAM – fast, less expensive, easy-to-write, volative
  - Used for stack and heap
  - Can be used for code and data section, if you choose
- ROM: Read Only Memory
  - Slower than RAM, cheaper than RAM, hard-to-write, non-volatile
  - Used for some code – many DriverLib functions are in ROM
    - The MAP_xxx mystery: When you download code from the resource explorer, it is full of functions that begin with MAP_; these functions are stored in ROM
- Flash
  - Best of both worlds, RAM vs. ROM
  - Slower than RAM, cheaper than RAM, between RAM and ROM for ease of writing, non-volatile
  - Used for code when you "flash" the program
  - Used to store the Interrupt Vector Table

MSP432 Memory Map

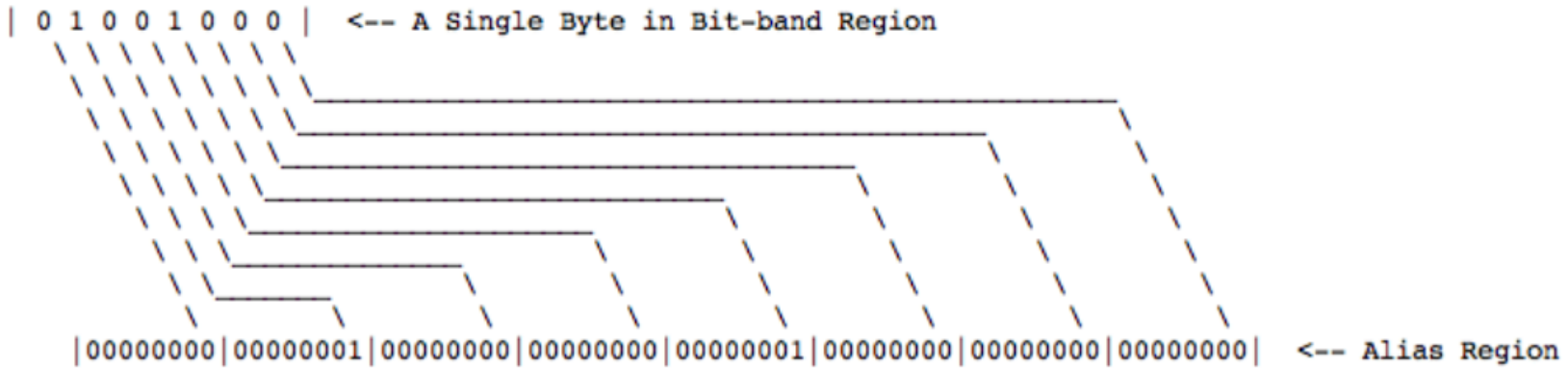


Bit-Banded

What is bit-banded?
https://spin.atomicobject.com/2013/02/08/bit-banding/

- Typically a CPU core cannot write to individual bits of a register; it must write a byte at a time
- To change a bit
  1. Read the current value of the register into a temporary register
  2. Modify the value of the temporary register with a logic operation
  3. Write the modified temporary register back to the register
- This is known as Read-Modify-Write
- Things can happen during these three steps, like interrupts, that can create undiserable effects

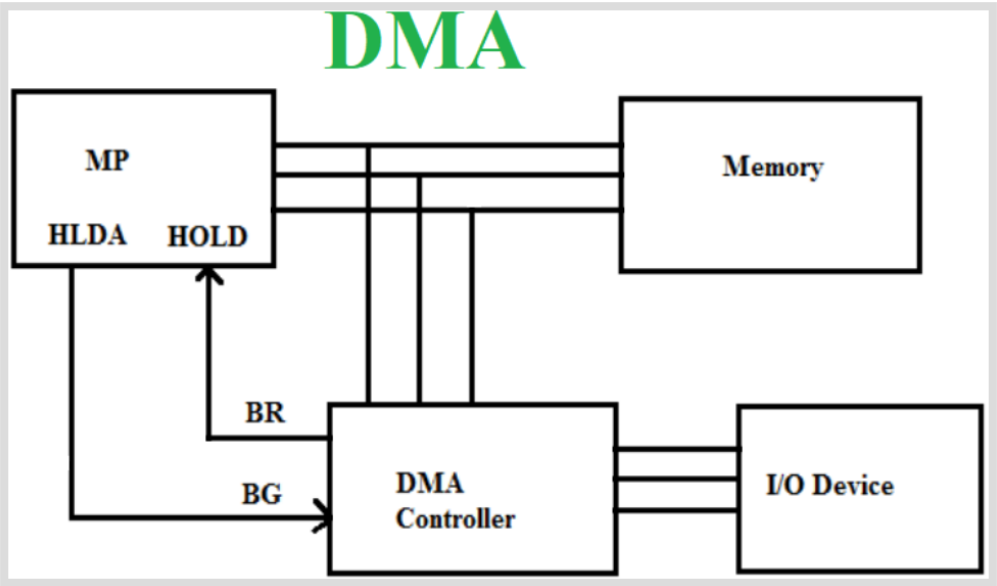
Bit-Banding in More Detail



- A region of memory, *the Bit-band region* maps each bit in that region to an entire word in another region of memory, *the Bit-Band alias region*
- Writing a word to the alias region performs a write to corresponding bit in the Bit-band region
- Reading a word in the alias region returns the corresponding bit in the Bit-band region

DMA: Direct Memory Access

- Transfer large chunks of data from memory to/from another device *without* getting the CPU involved in the transfer of every byte
  - DMA system send a request to CPU to transfer data
  - CPU grants DMA access
    - CPU can work on something else while the DMA is transferring data
  - DMA now transfers the data from Memory to the IO device



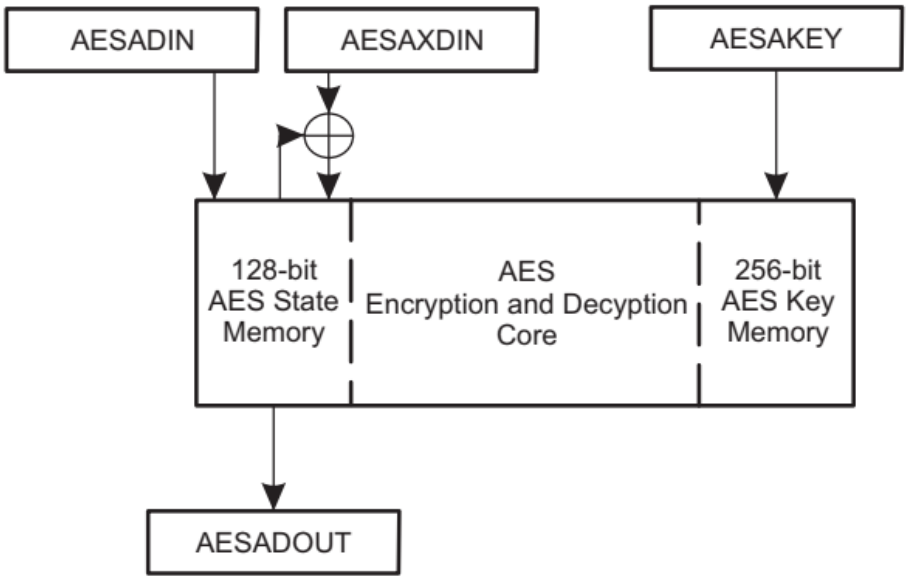
- See Chapter 11 of the User Manual
- Software examples are under Resource Explorer on DMA (in DriverLib)

RTC: Real-Time Clock

- See Chapter 20 of the User Manual
- Can turn the MSP432 into a real-world clock, including alarms
  - Set today's date and it will start
  - Can set interrupts to trigger every minute, for example

AES: Advanced Encryption Standard

- Security engine standardized by NIST
  - Turn *plaintext* into *ciphertext*
  - Public key versus private key
- See Chapter 16 of the User Manual



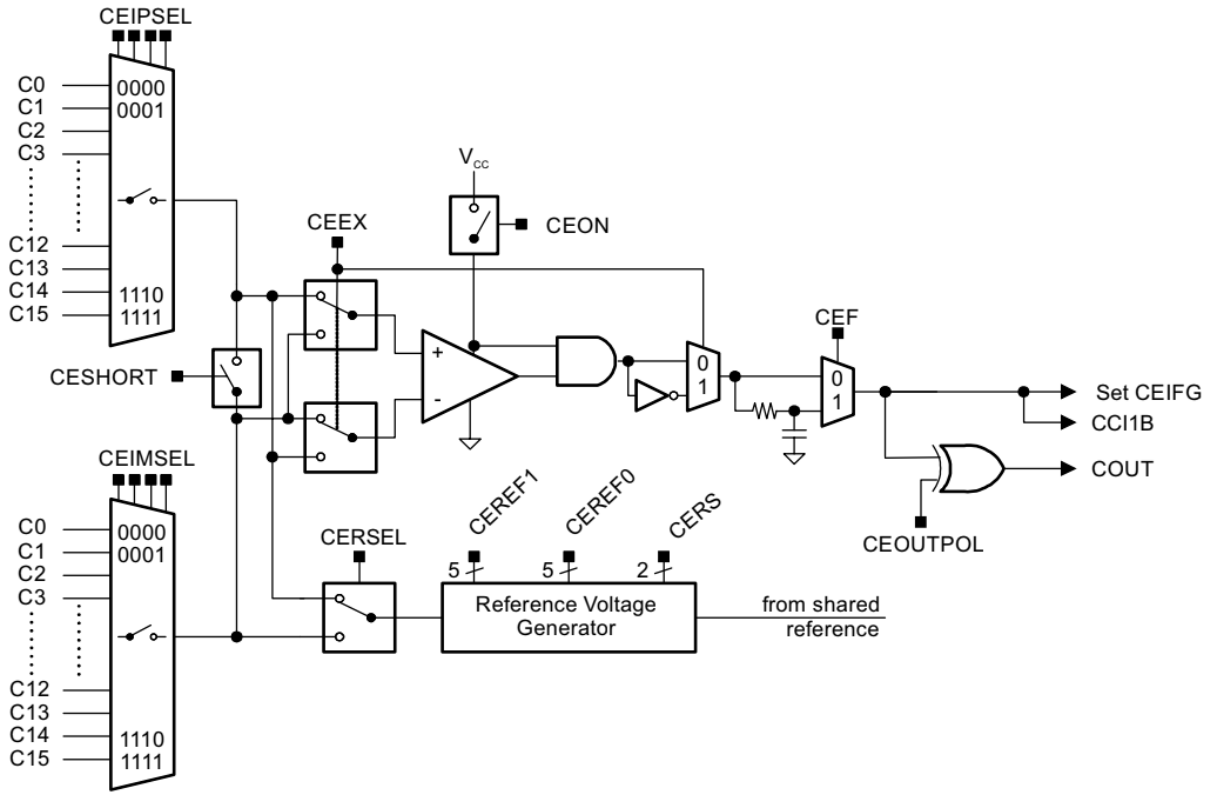
AES Accelerator Block Diagram

CRC: Cyclic Redundancy Check

- An approach for detecting data errors
- Much more sophisticated than parity
- Analogous to cyclically computing the remainder by dividing the input string by a number
  - Treat input string as a million-bit number
  - Remainder after division is the CRC

Comparator_E

- Performs analog voltage comparisons for upto 16 channels
- See Chapter 23 of the User Manual



COMP_E Block Diagram

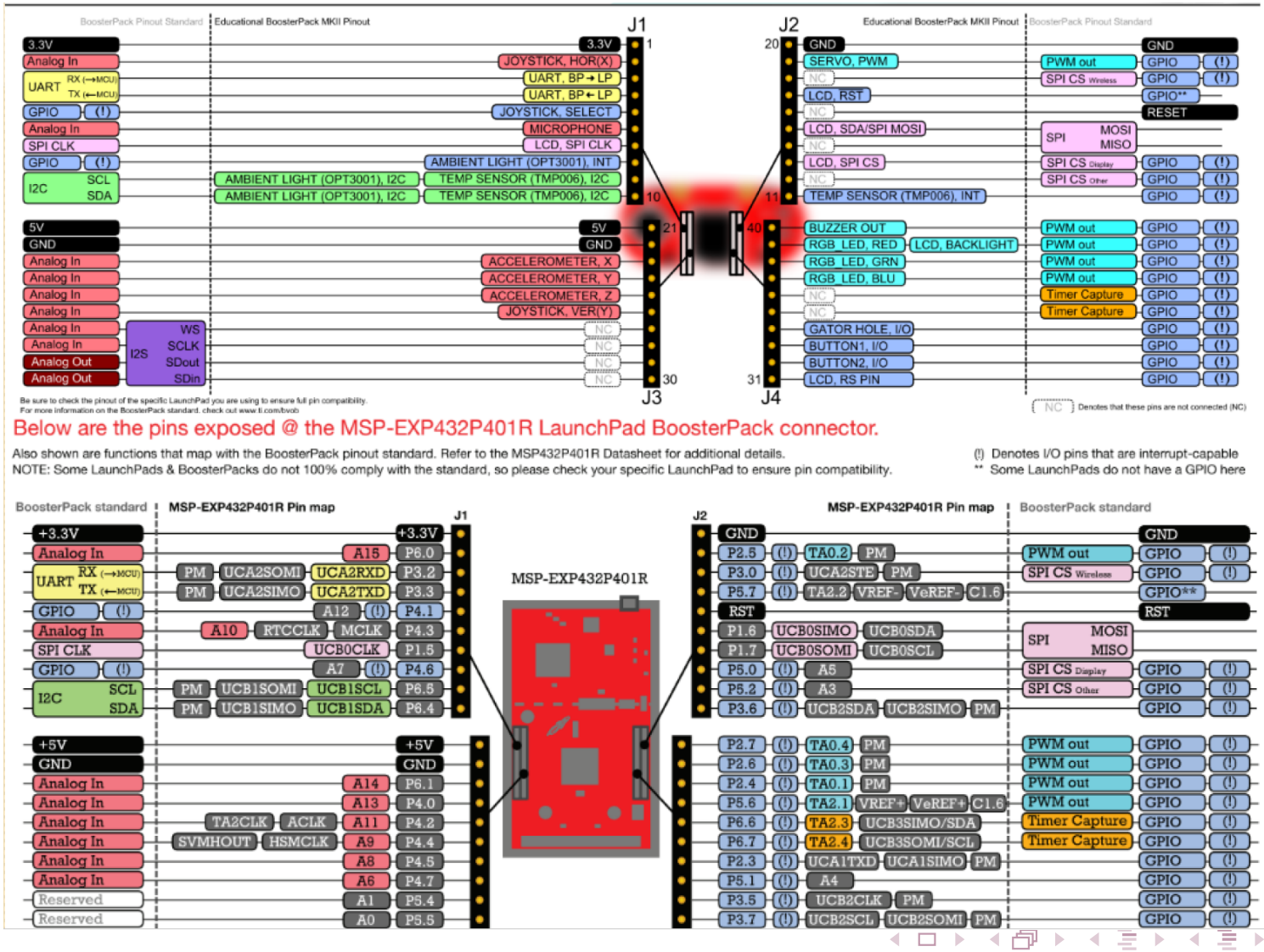
Ref_A

- Used for generating voltage references
  - Generate various *analog* voltages needed by peripherals, such as the ADC reference voltages
- See Chapter 21 of the User Manual

TI BoosterPack

- What we have covered
  - LEDs - GPIO
  - Buttons - GPIO
  - Joystick - ADC
  - Buzzer - Timer_A
  - Temperature and light sensor - I^2C
  - LCD - SPI
- What is left?
  - Microphone - ADC
    - Contains a Fast Fourier Transform (FFT) engine (signal processor)
  - Accelerometer - ADC
    - Interfacing is similar to the joystick

BoosterPack Pinout Diagram



Summary

- Bare-metal programming versus RTOS
- Quick overview of the remaining modules on our MSP432 kits
  - MSP432 microcontroller
  - LaunchPad board
  - BoosterPack board
- Those other modules can be programmed much like the modules that we have studied